

Motor de Verificação de Tipos

Relatório Técnico — Projeto de Compiladores

Disciplina: Construção de Compiladores

Linguagem utilizada: Python 3.8+

Introdução

A verificação de tipos é uma das etapas mais críticas no processo de análise semântica de um compilador. Ela garante que as operações realizadas sobre dados sejam compatíveis com os tipos declarados ou inferidos pelo sistema, prevenindo erros em tempo de execução e produzindo mensagens de diagnóstico claras ao programador.

Este projeto implementa um **Motor de Verificação de Tipos** em Python que avalia expressões matemáticas representadas no formato JSON. O motor simula dois mecanismos fundamentais presentes em linguagens de programação modernas: a **coerção implícita** (conversão automática entre tipos compatíveis, como *int* para *float*) e o **cast explícito** (conversão forçada declarada pelo programador).

O sistema também mantém uma **tabela de símbolos**, estrutura essencial em compiladores para armazenar informações sobre variáveis declaradas no escopo atual, incluindo nome, tipo e valor.

O objetivo central do trabalho é aplicar, de forma prática, os conceitos de análise semântica, equivalência de tipos e propagação de erros estudados em sala, demonstrando como um compilador real tomaria decisões de tipo durante a fase de análise de um programa-fonte.

Metodologia de Implementação

2.1 Estrutura do Projeto

O projeto foi dividido em três módulos Python com responsabilidades bem definidas:

Arquivo	Responsabilidade
type_checker.py	Define o enum Type, a classe SymbolTable (hash map), as regras de coerção implícita e as regras de cast explícito.

evaluator.py	Percorre recursivamente os nós JSON da expressão e retorna o valor avaliado, o tipo inferido e notas de coerção.
main.py	Entry point: lê expressions.json, invoca o avaliador e imprime o relatório formatado no terminal.
expressions.json	Arquivo de casos de teste com 8 expressões cobrindo todos os cenários relevantes.

2.2 Representação de Tipos (type_checker.py)

Os tipos suportados pelo motor são definidos como um **Enum**: *INT*, *FLOAT*, *BOOL*, *STRING* e *ERROR* (estado sentinela para propagação de falhas). As regras de coerção implícita são armazenadas em um dicionário Python — equivalente a um hash map — cujas chaves são tuplas (*tipo_origem*, *tipo_destino*) e cujos valores correspondem ao tipo resultante da operação:

```
COERCION_RULES = {
    (Type.INT, Type.FLOAT): Type.FLOAT,    # int op float -> float
    (Type.FLOAT, Type.INT): Type.FLOAT,    # float op int -> float
    (Type.INT, Type.BOOL): Type.INT,       # int op bool -> int
    (Type.BOOL, Type.FLOAT): Type.FLOAT,   # bool op float -> float
    ...
}
```

As regras de cast explícito são armazenadas em outro dicionário com valor booleano, indicando se a conversão é permitida. Por exemplo, *STRING* → *INT* é marcada como *False*, representando um cast inválido que o motor deve rejeitar com mensagem de erro.

2.3 Tabela de Símbolos (SymbolTable)

A classe **SymbolTable** encapsula um dicionário Python (*dict[str, Symbol]*) que mapeia nomes de variáveis a objetos **Symbol**, os quais carregam *nome*, *tipo* e *valor*. Os métodos *declare()* e *lookup()* simulam as operações de inserção e consulta típicas da tabela de símbolos de um compilador real.

2.4 Avaliador de Expressões (evaluator.py)

A função central **evaluate(node, sym_table)** implementa uma travessia recursiva da árvore de expressão representada em JSON. Cada chamada retorna um objeto **EvalResult** contendo o valor calculado, o tipo inferido, notas de coerção aplicadas e uma mensagem de erro caso a operação seja inválida.

Os tipos de nó reconhecidos pelo avaliador são:

Tipo de nó	Formato JSON	Comportamento
Literal	{"type": "int", "value": 3}	Retorna valor e tipo diretamente.
Variável	{"var": "x"}	Consulta a SymbolTable; erro se não declarada.
Operação binária	{"op": "+", "left": {...}, "right": {...}}	Avalia filhos; aplica coerção se necessário.
Cast explícito	{"op": "cast", "to": "float", "expr": {...}}	Verifica regra de cast; converte ou retorna erro.

A propagação de erros é feita por curto-circuito: ao detectar um *EvalResult.error* em qualquer subexpressão, o avaliador retorna imediatamente o erro sem continuar a avaliação, evitando exceções não tratadas e garantindo saídas previsíveis.

Casos de Teste

Os casos de teste foram definidos em *expressions.json* e executados com o comando **python main.py**. A seguir são apresentadas as entradas (nós JSON) e as saídas produzidas pelo motor.

Caso 1 — INT + INT (mesmo tipo)

Entrada (JSON):

```
{"op": "+", "left": {"type": "int", "value": 10},  
  "right": {"type": "int", "value": 5}}
```

Saída:

```
=> 15 (INT)
```

Operação entre tipos idênticos. Nenhuma coerção necessária.

Caso 2 — INT + FLOAT (coerção implícita)

Entrada (JSON):

```
{"op": "+", "left": {"type": "int", "value": 3},  
  "right": {"type": "float", "value": 2.5}}
```

Saída:

```
=> 5.5 (FLOAT) [coercao implicita: INT e FLOAT -> FLOAT]
```

INT é promovido a FLOAT automaticamente. A nota de coerção é registrada.

Caso 3 — Cast explícito FLOAT -> INT

Entrada (JSON):

```
{"op": "cast", "to": "int",  
  "expr": {"type": "float", "value": 9.7}}
```

Saída:

```
=> 9 (INT) [cast FLOAT -> INT]
```

Conversão válida; valor é truncado para 9 (comportamento de int()).

Caso 4 — Cast inválido STRING -> INT

Entrada (JSON):

```
{"op": "cast", "to": "int",  
  "expr": {"type": "string", "value": "hello"}}
```

Saída:

=> ERRO: Cast invalido: STRING -> INT

STRING -> INT é explicitamente proibido nas regras. Motor retorna erro sem execucao.

Caso 5 — Variável da SymbolTable (x * 2)

Entrada (JSON):

```
vars: [{"name": "x", "type": "int", "value": 7}]
{"op": "*", "left": {"var": "x"},
  "right": {"type": "int", "value": 2}}
```

Saída:

=> 14 (INT)

Variavel x e consultada na SymbolTable; operacao realizada com tipo INT.

Caso 6 — Divisão por zero

Entrada (JSON):

```
{"op": "/", "left": {"type": "int", "value": 8},
  "right": {"type": "int", "value": 0}}
```

Saída:

=> ERRO: Divisao por zero

Detectado antes da divisao; motor retorna EvalResult.error sem lancar execucao.

Caso 7 — Concatenação STRING + STRING

Entrada (JSON):

```
{"op": "+", "left": {"type": "string", "value": "Hello, "},
  "right": {"type": "string", "value": "World!"}}
```

Saída:

=> 'Hello, World!' (STRING)

Operador + e o unico permitido para STRING; resultado e a concatenacao.

Caso 8 — BOOL * FLOAT (coerção implícita)

Entrada (JSON):

```
{"op": "*", "left": {"type": "bool", "value": true},
  "right": {"type": "float", "value": 3.14}}
```

Saída:

=> 3.14 (FLOAT) [coercao implicita: BOOL e FLOAT -> FLOAT]

*BOOL e promovido a FLOAT (True=1.0). Resultado: 1.0 * 3.14 = 3.14.*

Todos os 8 casos foram executados com sucesso: os 5 casos válidos produziram valores e tipos corretos, e os 3 casos de erro retornaram mensagens descritivas sem lançar exceções não tratadas, demonstrando robustez do motor.